Hands-on Lab - CRUD operations with Node.js

Skills Network

Estimated Time Needed: 1 hour

In this lab you will learn how to create a **Friend's list** using Express server. Your application should allow you to add a friend with the following details: `First name`, `Last name`, `Email` and `Date of birth`. You will also be providing the application the ability to retrieve details, change details and delete the details.

You will be creating an application with API endpoints to perform Create, Retrieve, Update and Delete operations on the above data using an Express server.

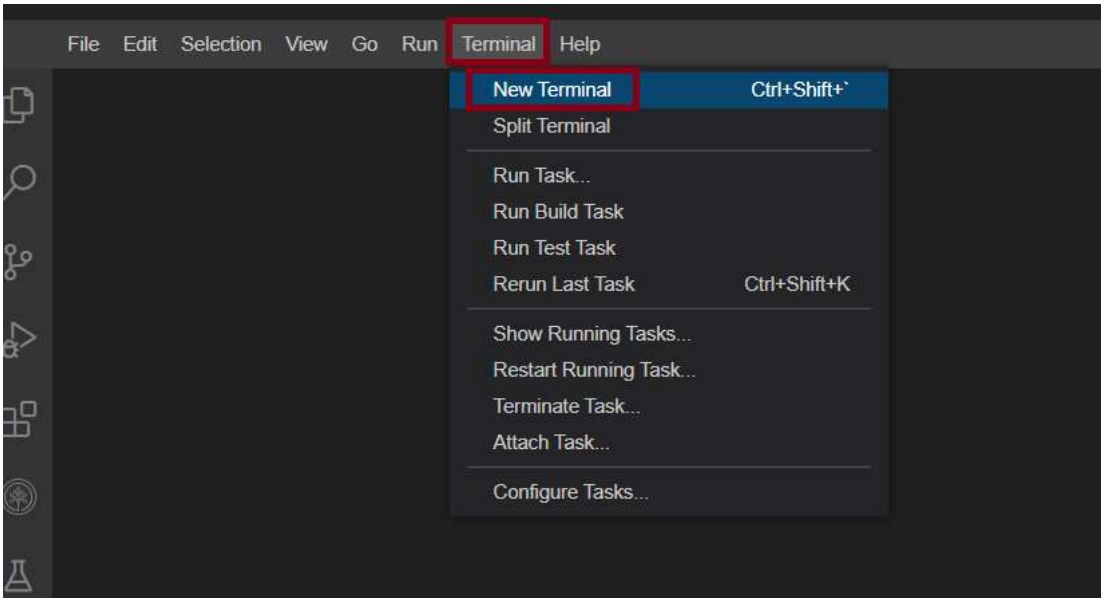
You will also learn to provide authenticated access to the endpoints. You will use cURL and Postman to test the implemented endpoints.

Objectives:

- Create API endpoints to perform Create, Retrieve, Update and Delete operations on transient data with an Express server.
- Implement authentication at the session level using JSON Web Tokens (JWT) for authorized access.

Set-up : Create application

1. Open a terminal window by using the menu in the editor: Terminal > New Terminal.



2. Change to your project folder, if you are not in the project folder already.

```
1
cd /home/project
```

Copied!

3. Run the following command to clone the git repository that contains the starter code needed for this lab, if it doesn't already exist.

```
1
[ ! -d 'mxpfu-nodejsLabs' ] && git clone https://github.com/ibm-developer-skills-network/mxpfu-nodejsLabs.git
```

Copied!

```
theia@theiadocke[redacted]/home/project$ [ ! -d 'mxpfu-nodejsLabs' ] && git clone https://github.com/ibm-develope
-nodejsLabs.git
Cloning into 'mxpfu-nodejsLabs'...
remote: Enumerating objects: 100, done.
remote: Counting objects: 100% (47/47), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 100 (delta 28), reused 21 (delta 13), pack-reused 53
Receiving objects: 100% (100/100), 55.52 KiB | 5.55 MiB/s, done.
Resolving deltas: 100% (36/36), done.
theia@theiadocke[redacted]/home/project$
```

5. Change to the directory **mxpfu-nodejsLabs** directory to start working on the lab.

```
1
cd mxpfu-nodejsLabs/
```

Copied!

6. List the contents of this directory to see the artifacts for this lab.

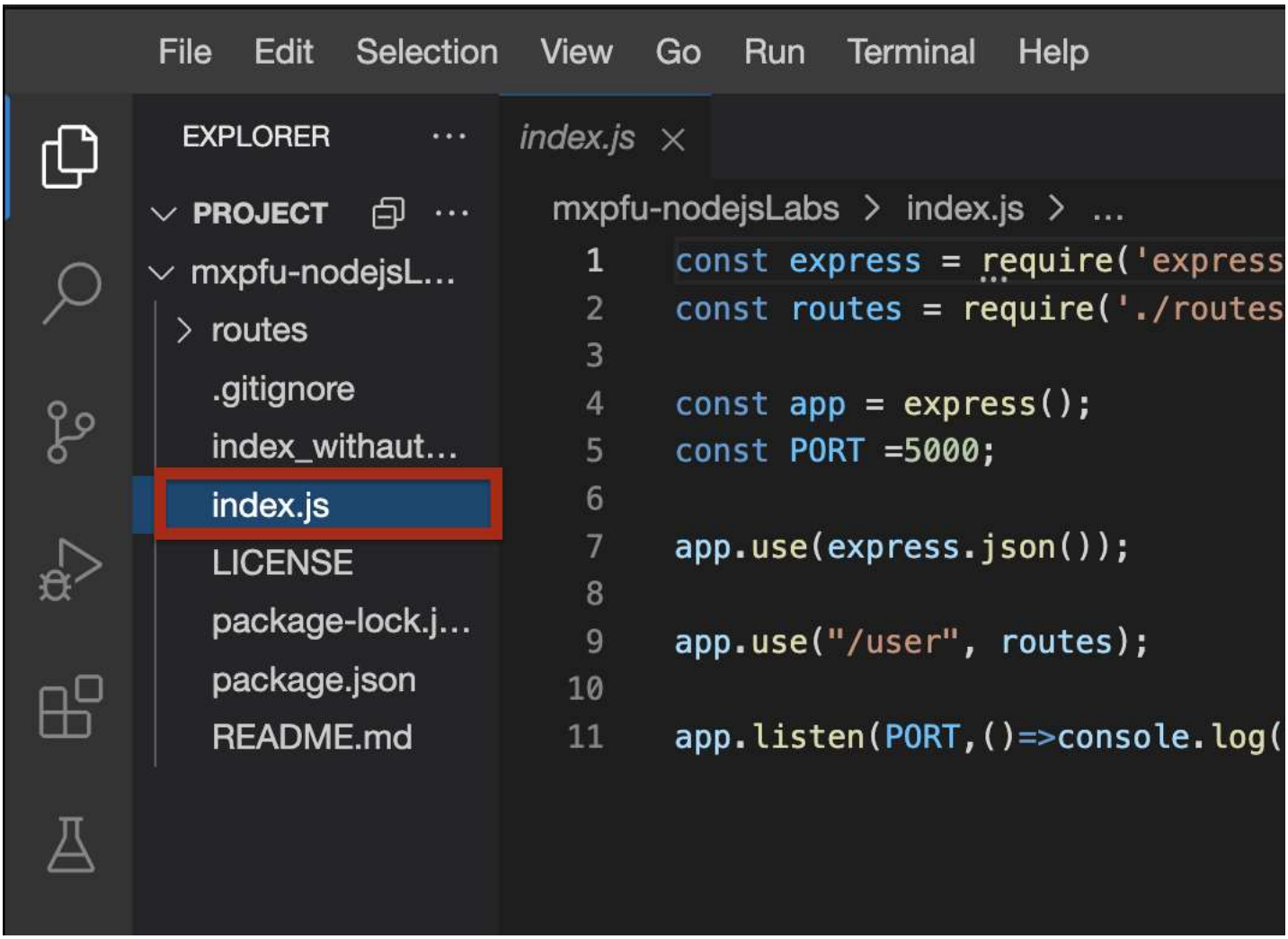
```
1
ls
```

Copied!

```
theia@theiadocker-lavanyas:/home/project/mxpfu-nodejsLabs$ ls
index.js            LICENSE        package-lock.json  routes
index_withauth.js   package.json   README.md
```

Exercise 1: Understand the server application

1. In the Files Explorer open the **mxpfu-nodejsLabs** folder and view **index.js**.



You have an Express server that has been configured to run at port 5000. When you access the server with **/user** you can access the endpoints defined in **routes/users.js**.

Recall that GET, POST, PUT and DELETE are the commonly used HTTP methods to perform CRUD operations. Those operations retrieve and send data to the server.

- `GET` is used to request data from a specified resource.
- `POST` is used to send data to a server for creating a resource.
- `PUT` is used to send data to a server to update a resource.
- `DELETE` is used for deleting a specified resource.

  `POST` AND `PUT` are sometimes used interchangeably.

2. This lab requires some packages to be installed. The **express** and **nodemon** package for starting and running the Express server and **jsonwebtoken** and **express-session** for session based authentication.

- **express** - This is for creating a server to serve the API endpoints.
- **nodemon** - This will help to restart the server when you make any changes to the code.
- **jsonwebtoken** - This package helps in generating a JSON web token which we will use for authentication. A **JSON web token (JWT)** is a JSON object used to communicate information securely over the internet (between two parties). It can be used for information exchange and is typically used for authentication systems.
- **express-session** - This package will help us to maintain the authentication for the session.

These packages are defined in as **dependencies** in packages.json.

```
1
2
3
4
5
6
```

```
  "dependencies": {
    "express": "^4.18.1",
    "express-session": "^1.17.3",
    "jsonwebtoken": "^8.5.1",
    "nodemon": "^2.0.19"
}
```

Copied!

3. Observe that the express app uses the middleware **express.json()** to handle the request as a json object.

```
1
```

```
app.use(express.json());
```

Copied!

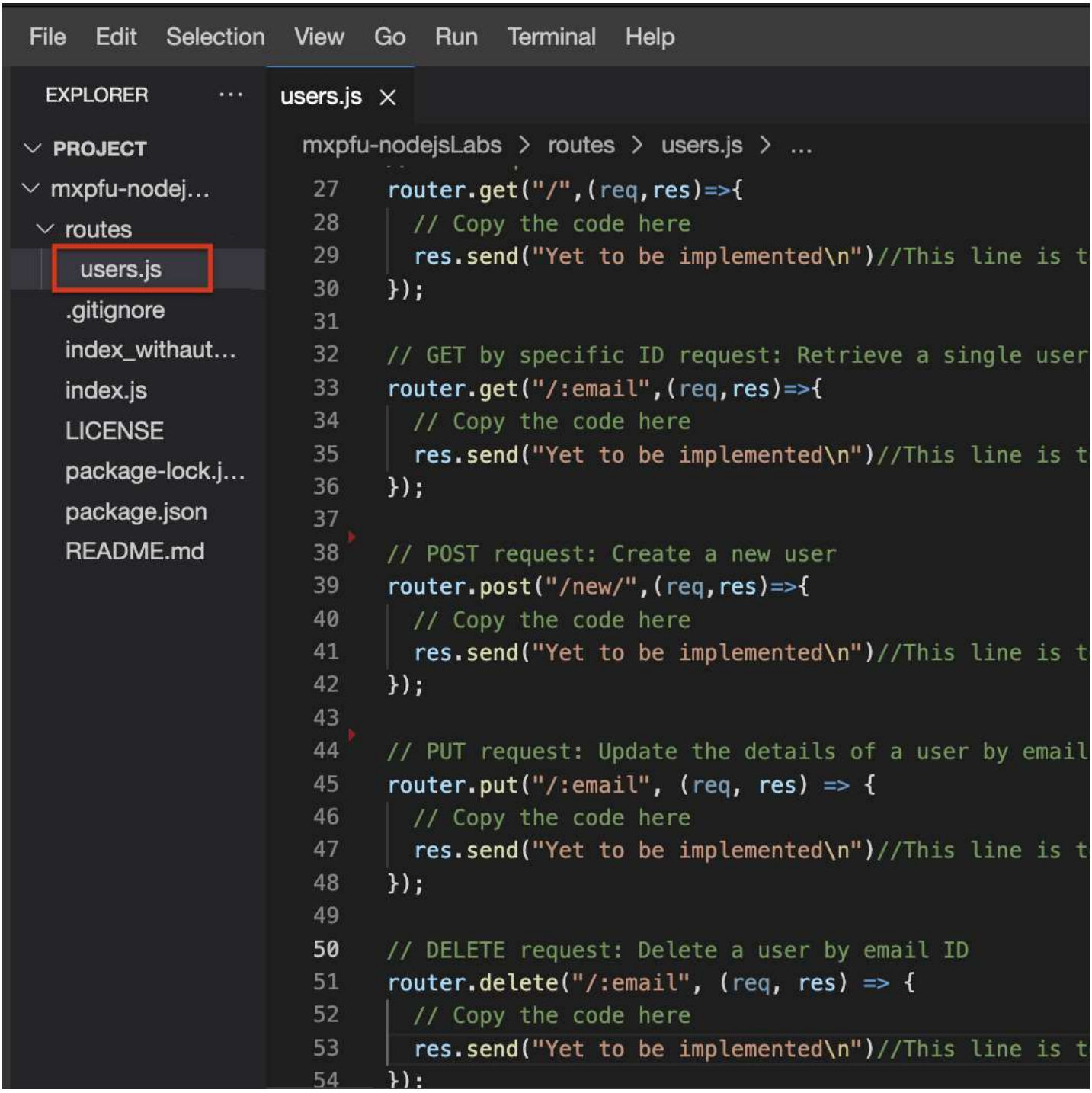
4. Observe that the express app uses routes to handle the endpoints which start with **/user**. This means that for all the endpoints starting with **/user**, the server will go and look for an endpoint handler in **users.js**.

```
1
```

```
app.use("/user", routes);
```

Copied!

5. All the endpoints have skeletal, but working implementation in **users.js**. Navigate to users.js under the directory routes and observe the endpoints defined in it.



Exercise 2: Run the server

The starter code given is a functioning server with dummy return values. Before starting to implement the actual endpoints, run the server.

1. In the terminal, print the working directory to ensure you are in **/home/projects/mxpfu-nodejsLabs**.

```
1
pwd
```

Copied!

2. Install all the packages that are required for running the server. Copy, paste, and run the following command.

```
1
npm install
```

Copied!

This will install all the required packages as defined in packages.json.

3. Start the express server.

```
1
```

```
npm start
```

Copied!

4. Open a **New Terminal** from the top menu. Test an endpoint to retrieve these users. This has not yet been implemented to return the users.

```
1
```

```
curl localhost:5000/user
```

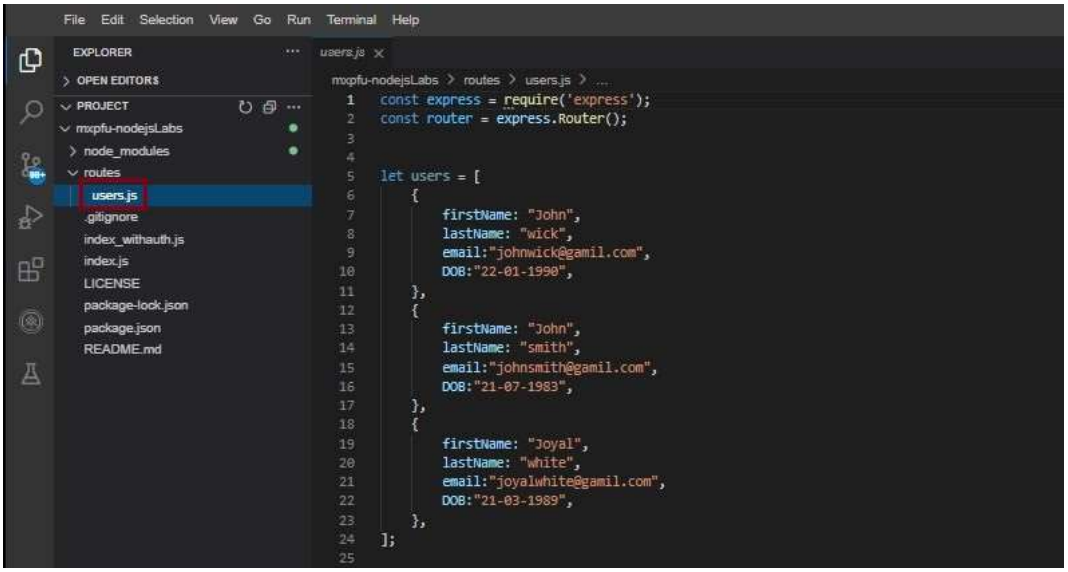
Copied!

```
theia@theiadocker-lavanyas:/home/project$ curl localhost:5000/user
Yet to be implemented
```

5. If you see the output as displayed above, it means the server is running as expected.

Exercise 2: Implement your endpoints

1. Navigate to the file named **users.js** in the **routes** folder. The endpoints have been defined and space has been provided for you to implement the endpoints.



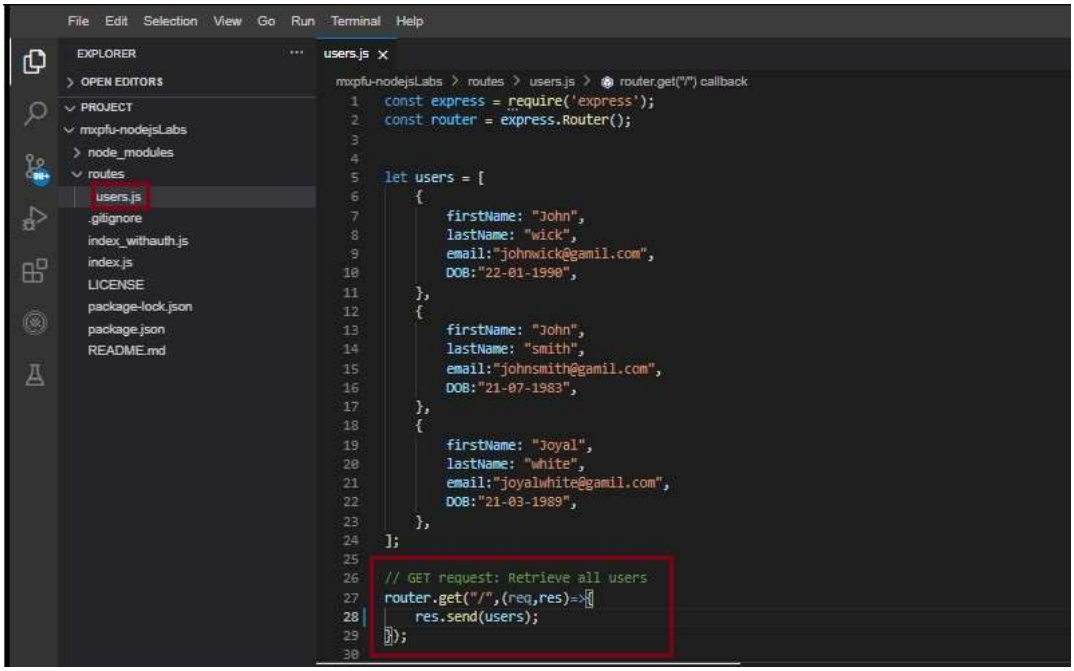
2. **R** in CRUD stands for retrieve. You will first add an API endpoint, using the **get** method for getting the details of all users. A few users have been added in the starter code.

- Copy the code below and paste in users.js inside the { } brackets within the **router.get("/ ",(req,res)=>{}** method.

```
1
```

```
    res.send(users);
```

Copied!

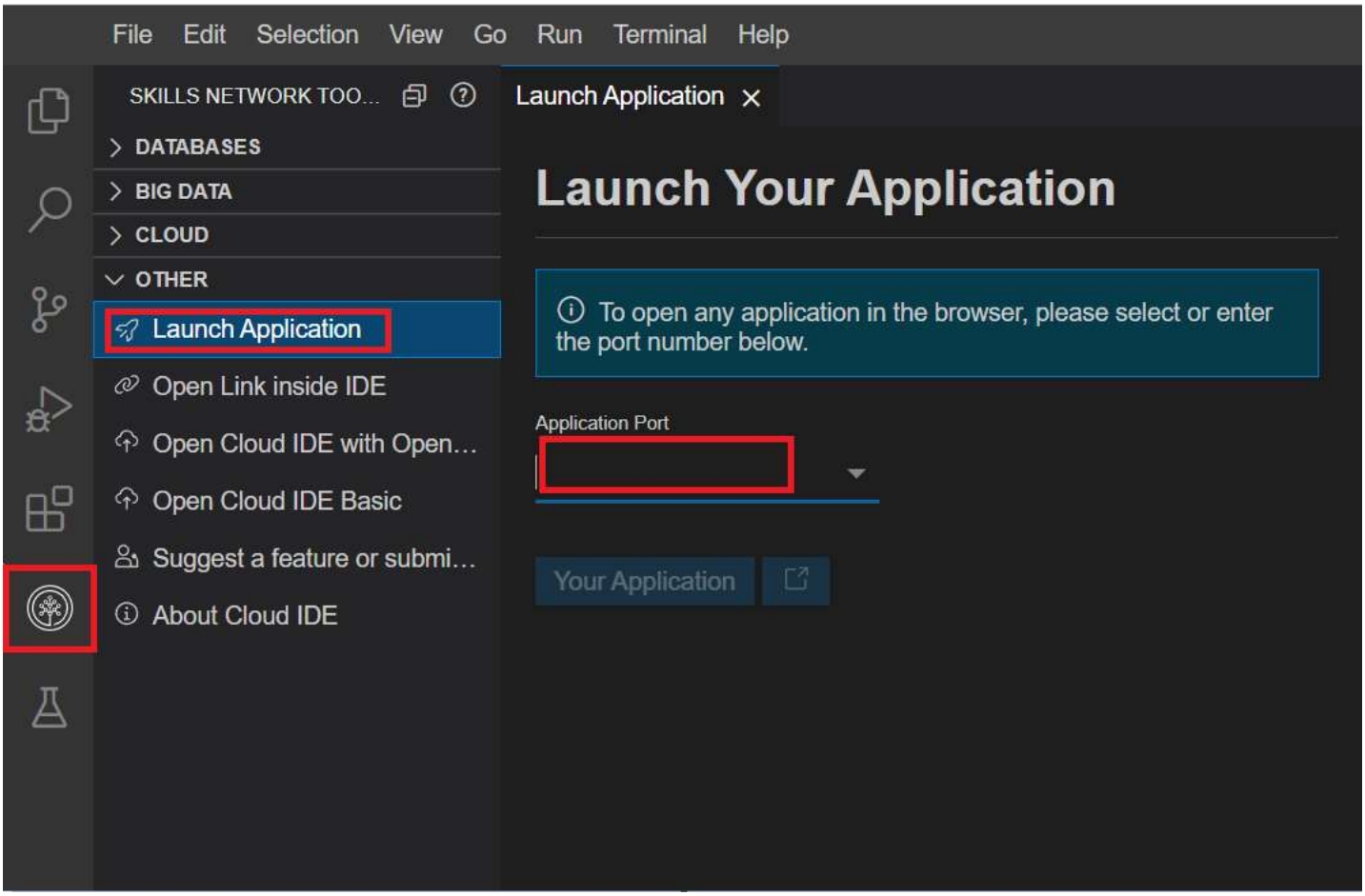


3. Ensure that your server is running. As you make changes to the code, the server that you started in the previous task, should be restart. If the server is not running, start it again.

```
1. l
1. npm start
```

Copied!

3. Click on the **Skills Network** button on the left. It will open the "Skills Network Toolbox". Then click `OTHER` then `Launch Application`. From there you should be able to enter the port as `5000` and launch the development server.



4. When the browser page opens up, suffix **/user** to the end of the URL on the address bar. You will see the below page.

```
[{"firstName":"John","lastName":"wick","email":"johnwick@gamil.com","DOB":"22-01-1990"},{"firstName":"John","lastName":"smith","email":"johnsmith@gam
{"firstName":"Joyal","lastName":"white","email":"joyalwhite@gamil.com","DOB":"21-03-1989"}]
```


5. Check the output of the GET request using the curl command just the way you did in the previous exercise.

```
1
curl localhost:5000/user/
```

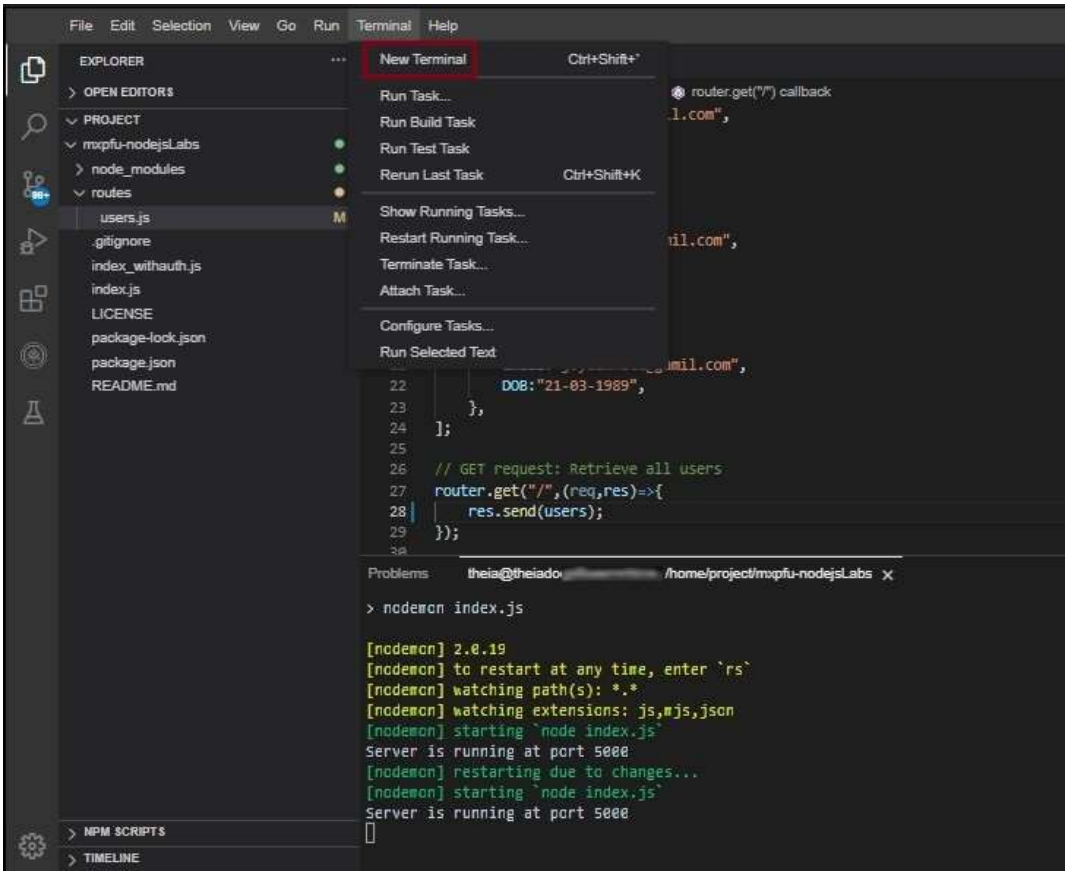
Copied!

Exercise 3: Creating a GET by specific email method:

1. Implement a **get** method for getting the details of a specific user based on their email ID by using the `filter` method on the user collection. Once you write the code and save it, the server will restart.

▶ Click here to view the code

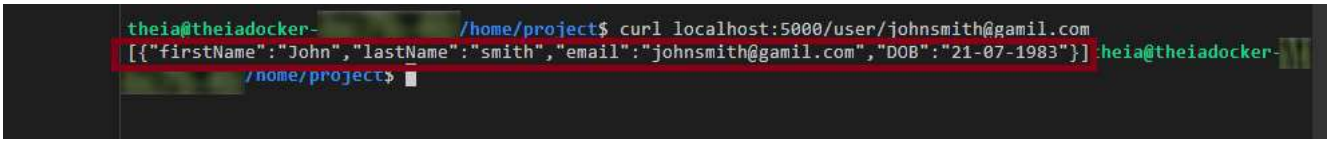
2. Click on Terminal > New Terminal



3. In the new terminal, use the below command to view the output for the user with mail id johnsmith@gamil.com

```
1
curl localhost:5000/user/johnsmith@gamil.com
```

Copied!



Exercise 4: Creating the POST method:

1. The **C** in CRUD stands for **Create**. Implement the **/user** endpoint with the POST method to create a user and add the user to the list. You can create the user object as a dictionary. You can use the sample user object displayed below.

```
1
2
3
```

```
4
5
6
```

```
{
    "firstName":"Jon",
    "lastName":"Lovato",
    "email":"jonlovato@theworld.com",
    "DOB":"10/10/1995"
}
```

Copied!

Use `push` to add the dictionary into the list of users. The user details can be passed as query paramters named *firstName, lastName, DOB and email*.

> Hint: Query param can be retrieved from the request object using request.query.paramname

▶ Click here to view the code

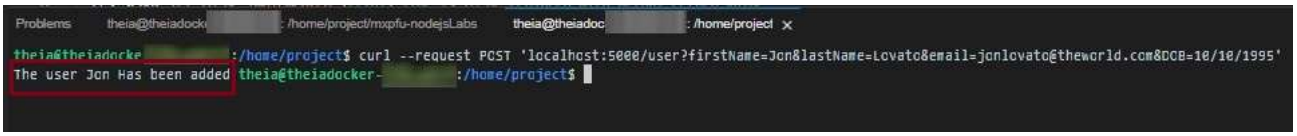
2. Use the below command to post a new user with mail id 'jonlovato@theworld.com' on the new terminal:

```
1
```

```
curl --request POST 'localhost:5000/user?firstName=Jon&lastName=Lovato&email=jonlovato@theworld.com&DOB=10/10/1995'
```

Copied!

3. The ouput will be as below:



4. To verify if the user with email 'jonlovato@theworld.com' has been added, you can send a GET request as below:

```
1
```

```
curl localhost:5000/user/jonlovato@theworld.com
```

Copied!

Exercise 5: Creating the PUT method:

1. The **U** in CRUD stands for update which can be achieved using the PUT method. To make updates in the data, you will use the PUT method. You should first look at the user with the specified email id and then modify it. The code below shows how the date of birth (DOB) of a user can be modified. Make the necessary code changes to allow changes to the other attributes of the user.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
```

```
router.put("/:email", (req, res) => {
    const email = req.params.email;
    let filtered_users = users.filter((user) => user.email === email);
    if (filtered_users.length > 0) {
        let filtered_user = filtered_users[0];
        let DOB = req.query.DOB;
        //if the DOB has changed
        if(DOB) {
            filtered_user.DOB = DOB
        }
        /*
        Include code here similar to the one above for other attibutes
        */
        users = users.filter((user) => user.email != email);
        users.push(filtered_user);
        res.send(`User with the email  ${email} updated.`);
```

```
    }
    else{
        res.send("Unable to find user!");
    }
});
```

Copied!

2. The completed code will look like this.

```
router.put("/:email", (req, res) => {
    const email = req.params.email;
    let filtered_users = users.filter((user) => user.email === email);
    if (filtered_users.length > 0) {
        let filtered_user = filtered_users[0];
        let DOB = req.query.DOB;
        let firstName = req.query.firstName;
        let lastName = req.query.lastName;
        if(DOB) {
            filtered_user.DOB = DOB
        }
        if(firstName) {
            filtered_user.firstName = firstName
        }
        if(lastName) {
            filtered_user.lastName = lastName
        }
        users = users.filter((user) => user.email != email);
        users.push(filtered_user);
        res.send(`User with the email  ${email} updated.`);
    }
    else{
        res.send("Unable to find user!");
    }
});


module.exports=router;
```

3. Use the below command to update the `DOB` as `1/1/1971` for the user with mail id 'johnsmith@gamil.com' in the split terminal:

```
1
```

```
curl --request PUT 'localhost:5000/user/johnsmith@gamil.com?DOB=1/1/1971'
```

Copied!

4. The ouput will be as below:

```
Problems    theia@theiad[redacted]: /home/project/mxpfu-nodejsLabs    theia@theiad[redacted]: /home/project
theia@theiadocke[redacted]:/home/project$ curl --request PUT 'localhost:5000/user/johnsmith@gamil.com?DOB=1/1/1971'
User with the email  johnsmith@gamil.com updated.theia@theiado[redacted]:/home/project$
```

5. To verify if the `DOB` of the user with email 'johnsmith@gamil.com' has been updated, you can send a GET request as below:

```
1
```

```
curl localhost:5000/user/johnsmith@gamil.com
```

Copied!

```
[nodemon] starting `node index.js`
Server is running
```

```
theia@theiadocker-[redacted]/home/project$ curl l
[{"firstName":"John","lastName":"smith","email":"joh
ome/project$
```

Exercise 6: Creating the DELETE method:

1. The "D" in CRUD stands for **Delete**. Implement the DELETE method for deleting a specific user's email by using the below code:

```
1
2
3
4
5
```

```
router.delete("/:email", (req, res) => {
    const email = req.params.email;
    users = users.filter((user) => user.email != email);
    res.send(`User with the email  ${email} deleted.`);
});
```

Copied!

2. The completed code will look like this.

```
        }
        users = users.filter((user) => user.email != email);
        users.push(filtered_user);
        res.send(`User with the email  ${email} updated.`);
    }
    else{
        res.send("Unable to find user!");
    }
  });

  router.delete("/:email", (req, res) => {
    const email = req.params.email;
    users = users.filter((user) => user.email != email);
    res.send(`User with the email  ${email} deleted.`);
  });


  module.exports=router;
```

3. Use the below command to delete the user with mail id 'johnsmith@gamil.com' in the split terminal:

```
1. 1
```

```
1. curl --request DELETE 'localhost:5000/user/johnsmith@gamil.com'
```

Copied!

4. The ouput will be as below:

```
Problems    theia@theiadock[redacted]: /home/project/mxpfu-nodejsLabs    theia@theiadock[redacted]: /home/project
theia@theiadocker-[redacted]:/home/project$ curl --request DELETE 'localhost:5000/user/johnsmith@gamil.com'
User with the email  johnsmith@gamil.com deleted.theia@theiadocker-[redacted]:/home/project$
```

5. Send a GET request for the user with email 'johnsmith@gamil.com' and ensure that a null object is returned:

```
theia@theiadocker[redacted]/home/project$ curl localhost:5000/user/curl localhost:5000/user/johnsmith@gamil.co
m
[][]theia@theiadocker-[redacted]home/project$
```

Optional Exercise: Formatting the output

1. To make the output more readable, you can use the JSON stringify method as given below. Please update the code for the GET method to:

```
1. 1
2. 2
3. 3
```

```
1. router.get("/",(req,res)=>{
2.     res.send(JSON.stringify({users},null,4));
3. });
```

Copied!

2. Launch the app on port 5000 & append 'user' to the end of the URL.

3. This will render the output of the GET method as a JSON string per the updated GET method shown below:

```
[redacted]-5000.theiadocker-2-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user

{ "users": [ { "firstName": "John", "lastName": "wick", "email": "johnwick@gamil.com", "DOB": "22-01-1990" }, { "firstName": "
"Joyal", "lastName": "white", "email": "joyalwhite@gamil.com", "DOB": "21-03-1989" } ] }
```

Exercise 7: Implementing Authentication

All these endpoints are accessible by anyone. You will now see how to add authentication to the CRUD operations. This code has been implemented in **index_withauth.js**.

1. Observe the following code block in **index_withauth.js**.

```
1
```

```
app.use(session({secret:"fingerprint",resave: true, saveUninitialized: true}))
```

Copied!

This tells your express app to use the session middleware.

- **secret** - a random unique string key used to authenticate a session.
- **resave** - takes a Boolean value. It enables the session to be stored back to the session store, even if the session was never modified during the request.
- **saveUninitialized** - this allows any uninitialized session to be sent to the store. When a session is created but not modified, it is referred to as **uninitialized**.

The default value of both **resave** and **saveUninitialized** is true, but the default is deprecated. So, set the appropriate value according to the use case.

2. Observe the implementation of the **login** endpoint. A user logs into the system providing a username. An access token that is valid for one hour is generated. You may observe this validty length specified by **60 * 60**, which signifies the time in seconds. This access token is set into the session object to ensure that only authenticated users can access the endpoints for that length of time.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
app.post("/login", (req,res) => {
    const user = req.body.user;
    if (!user) {
        return res.status(404).json({message: "Body Empty"});
    }
    let accessToken = jwt.sign({
        data: user
      }, 'access', { expiresIn: 60 * 60 });

      req.session.authorization = {
        accessToken
    }
    return res.status(200).send("User successfully logged in");
});
```

Copied!

3. Observe the implementation of the authentication middleware. All the endpoints starting with **/user** will go through this middleware. It will retrieve the authorization details from the session and verify it. If the token is validated, the user is authenticated and the control is passed on to the next endpoint handler. If the token is invalid, the user is not authenticated and an error message is returned.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
```

```
app.use("/user", (req,res,next)=>{
// Middleware which tells that the user is authenticated or not

   if(req.session.authorization) {
       let token = req.session.authorization['accessToken']; // Access Token

       jwt.verify(token, "access",(err,user)=>{
           if(!err){
               req.user = user;
               next();
           }
           else{
               return res.status(403).json({message: "User not authenticated"})
```

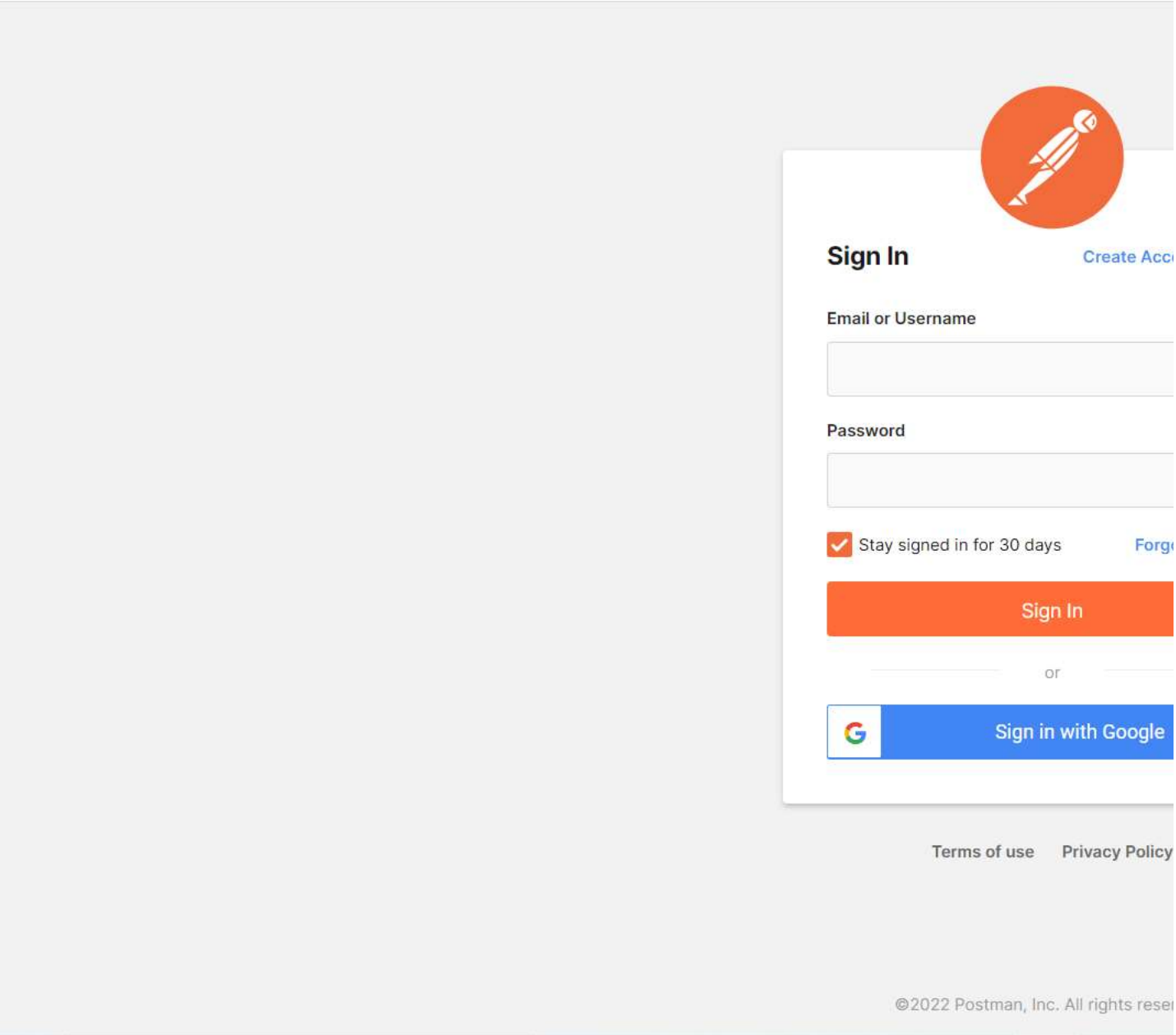
```
            }
        });
    } else {
        return res.status(403).json({message: "User not logged in"})
    }
});
```

Copied!

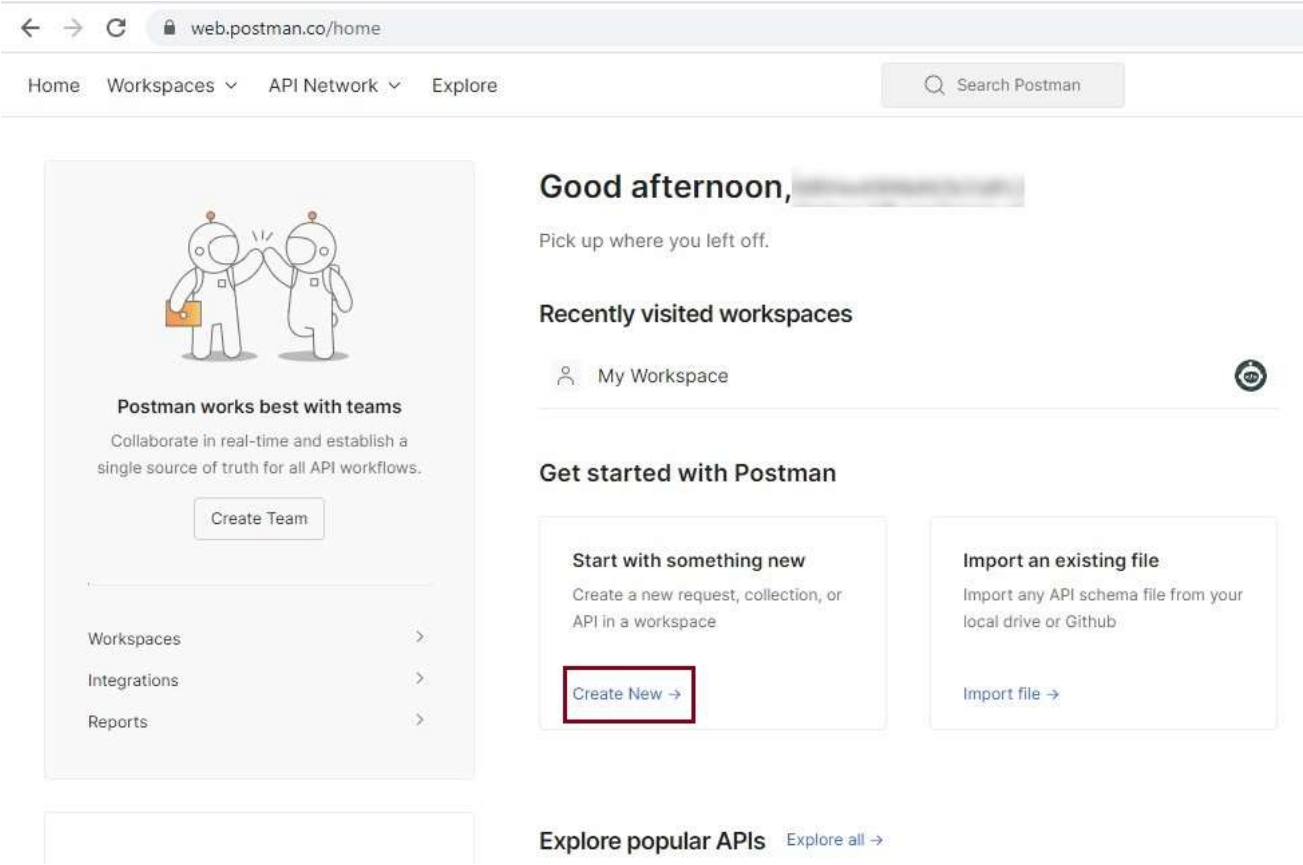
Exercise 8: Testing endpoints with POSTMAN

You have tested the API endpoints with cURL. An easier and more user-friendly way to test these endpoints with the graphical user interface tool (GUI), Postman.

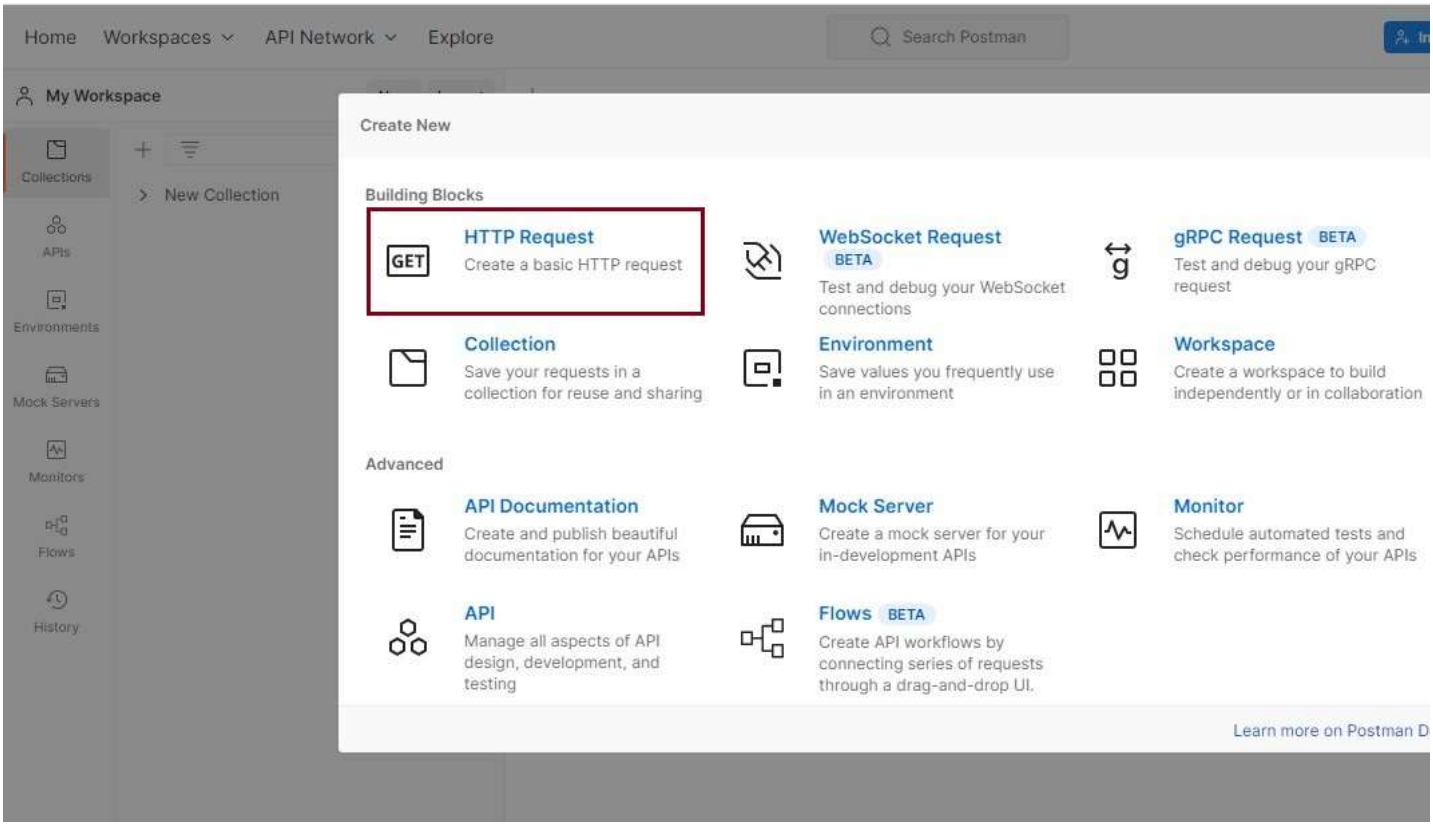
1. Go to [Postman](). Sign-up for a new Postman account if you don't already have one. Sign-in to your account.



2. After you login to Postman, click on **Create New** as shown below:



3. Select HTTP request from the pop-up window that appears.



Note: If the server is running in the theia lab please stop the server by press **CTRL + C**. Now start the server by running the below command which will listen to port 5000.

```
1
npm run start_auth
```

Copied!

So far we were accessing all the endpoints without authentication but now we will be using authentication to access the endpoints.

2. Copy the URL from the Launch application and add the login as an endpoint to add the user details in the **POST REQUEST** which will look like below:

```
1
```

```
https://<sn-lab-username>-5000.theiadocker-2-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/login
```

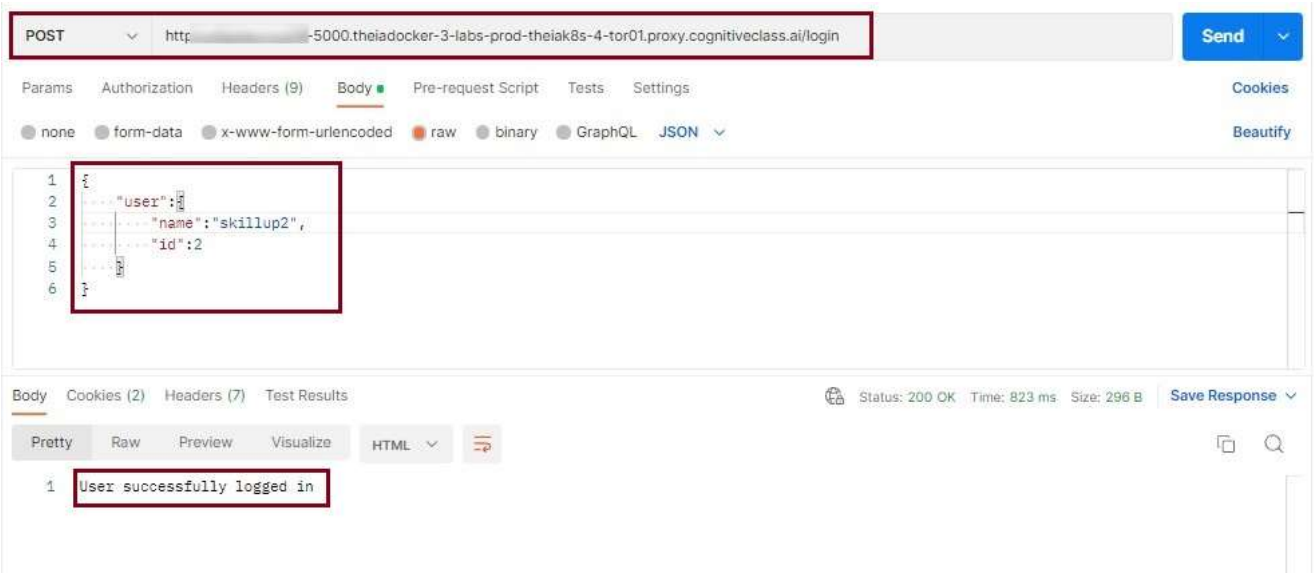
Copied!

3. User details should be in the below format:

```
1
2
3
4
5
6
```

```
{
    "user":{
        "name":"abc",
        "id":1
    }
}
```

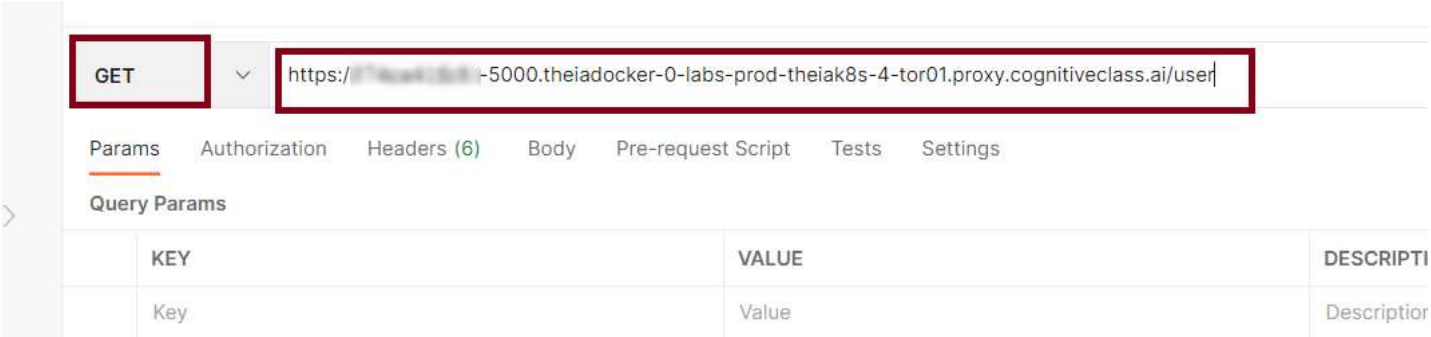
Copied!



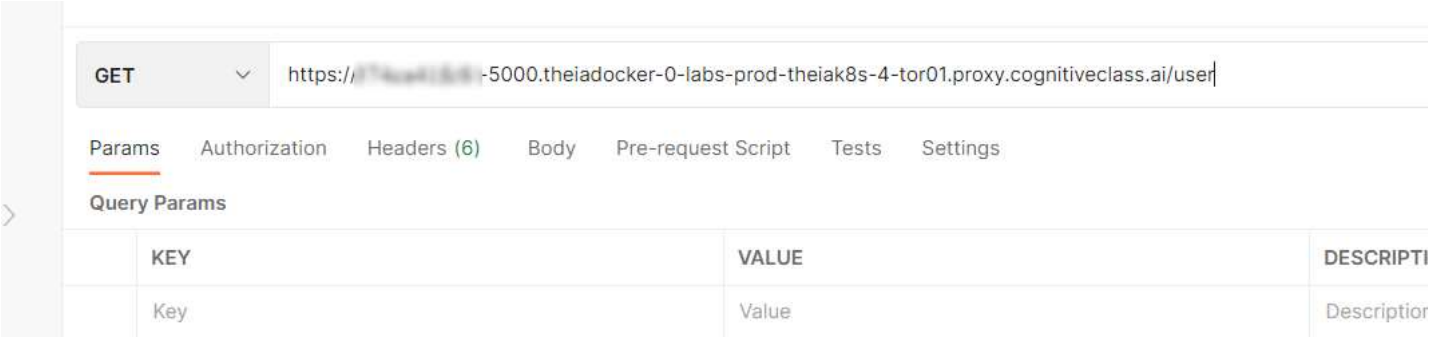
Now let's begin the test by sending an HTTP GET Request.

8.1 GET request

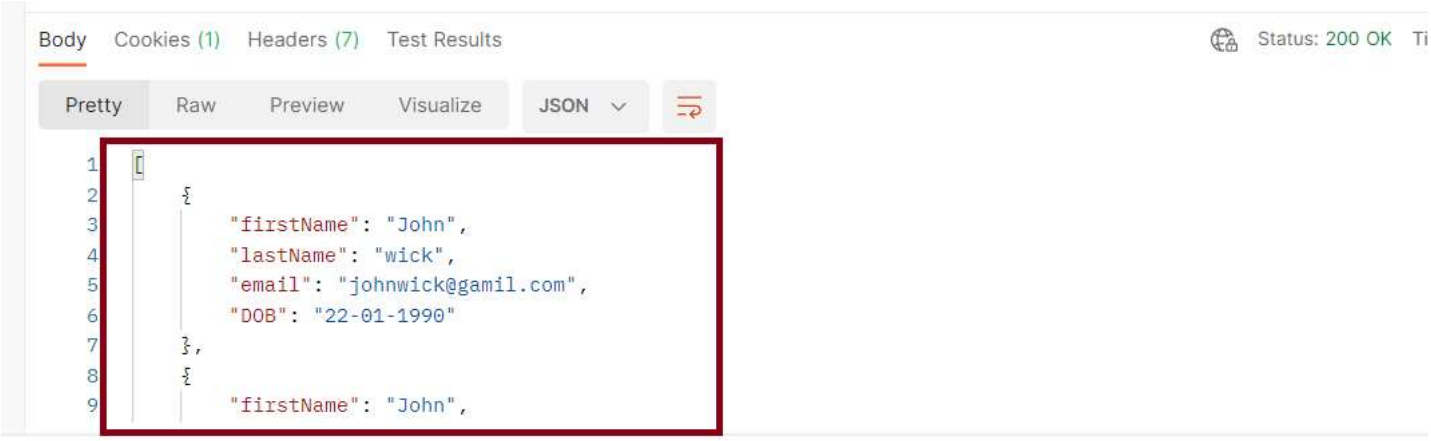
a. Enter the GET request URL: `https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user` in your input box of Postman where you see "Enter Request URL".



b. Click on the “Send” button after entering the URL.



c. The output will be as below:

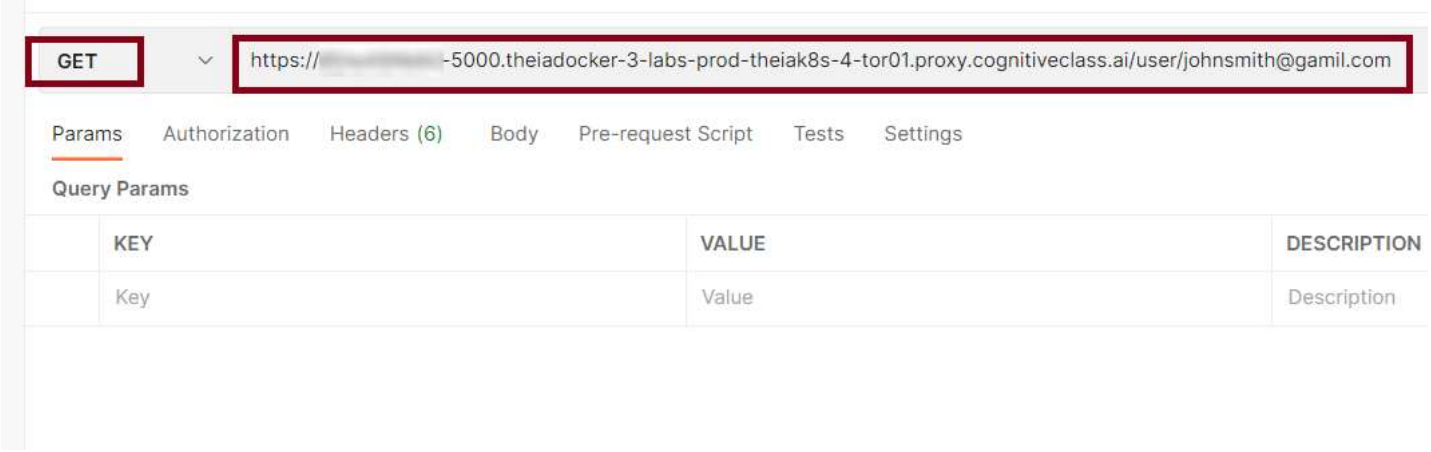


8.2 GET request by specific ID

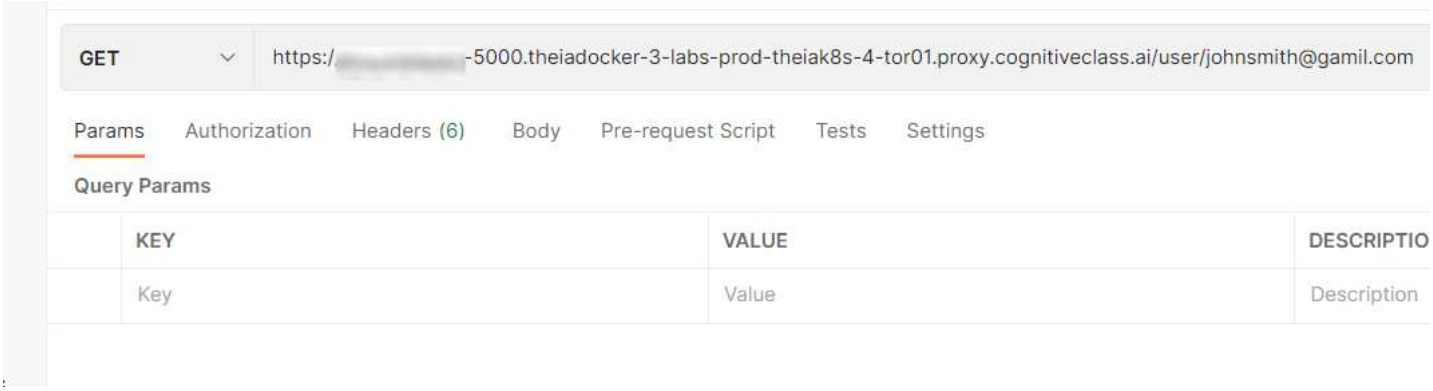
a. Enter the request URL by adding the specific email address to the above GET request URL. If the email address is johnsmith@gamil.com then enter the following URL in the input box of postman:

```
1. 1
1. https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user/johnsmith@gamil.com
```

Copied!



b. Click on the **Send** button after entering the URL to view the output.



c. The output will be as below:

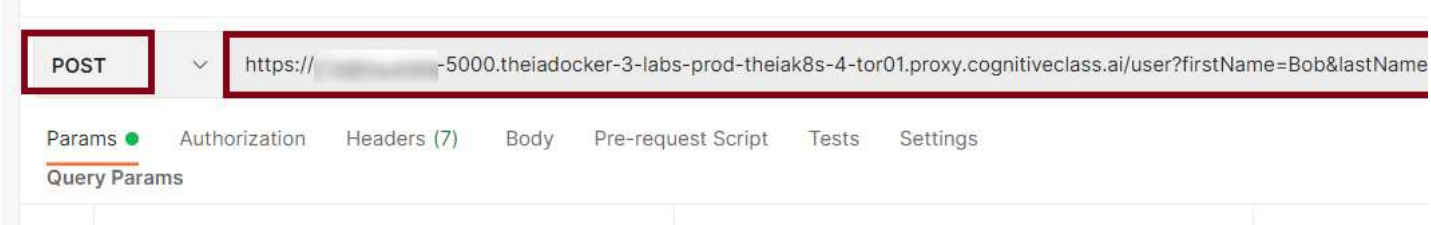
8.3 POST request :

a. Enter the basic post request URL:

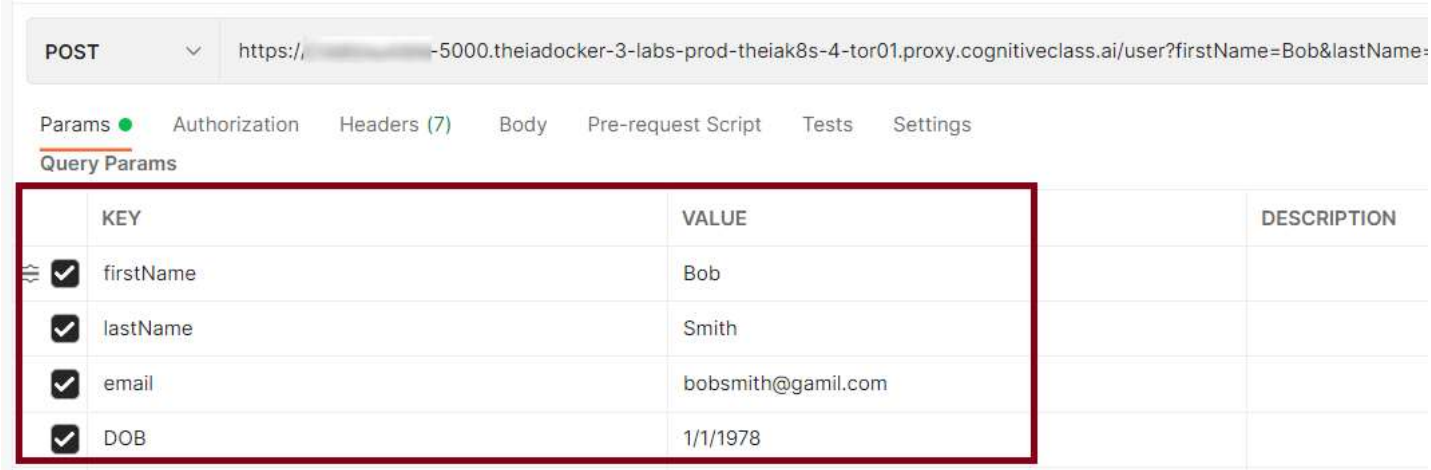
```
1. 1
1. https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user/
```

Copied!

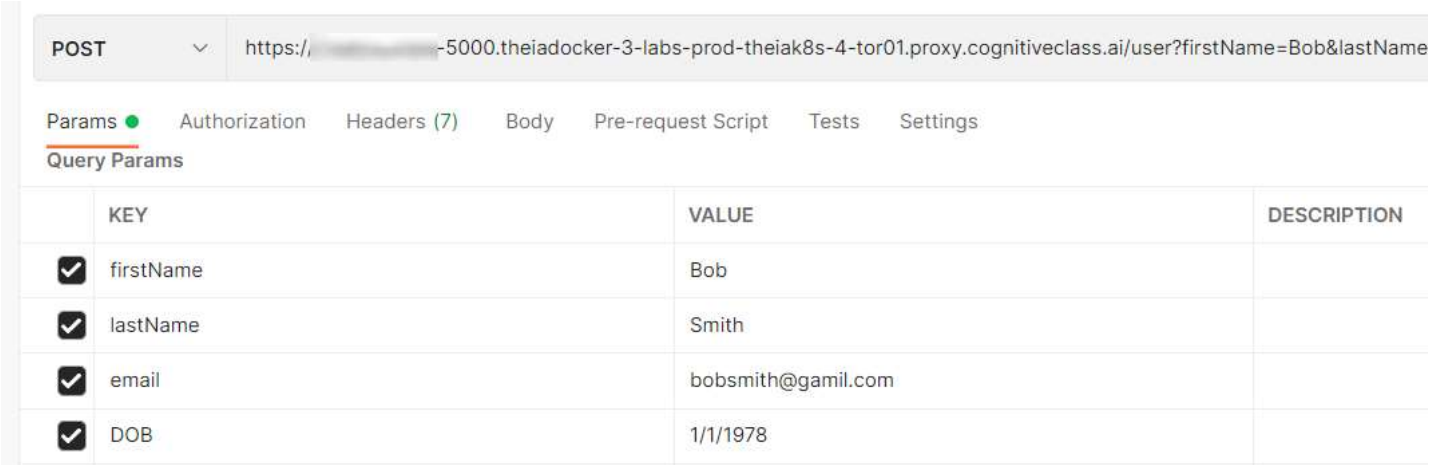
Ensure to select the POST method and select the "Params".



b. Enter the firstName as 'Bob', lastName as 'Smith', email as 'bobsmith@gamil.com' and DOB as '1/1/1978' for a new user:

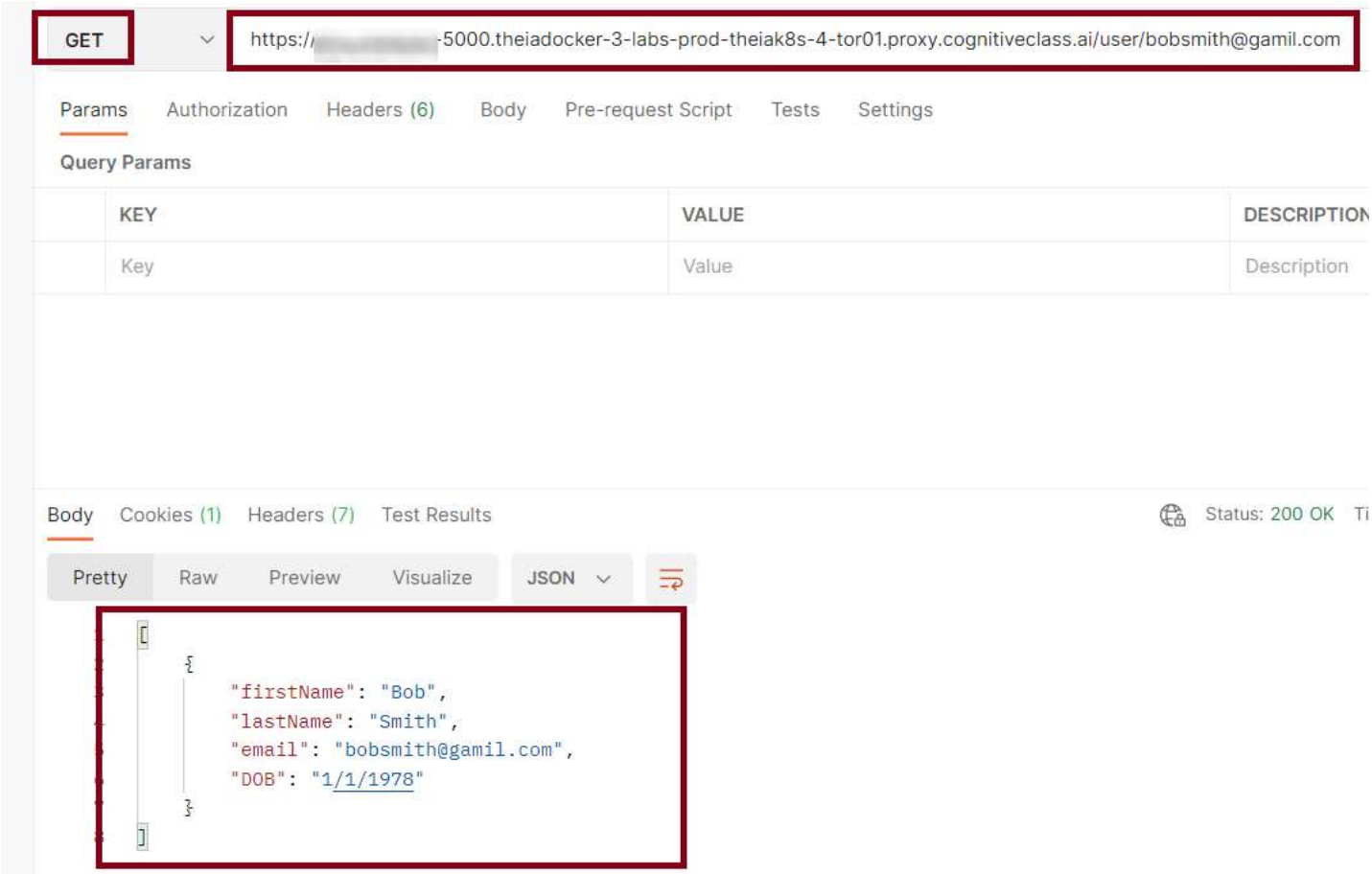


c. Click on the **Send** button after entering the URL to view the output.



Verify that the newly added values have been updated by doing the GET request.

Note: Ensure that you delete any parameters that you added for the POST request before sending the GET request.



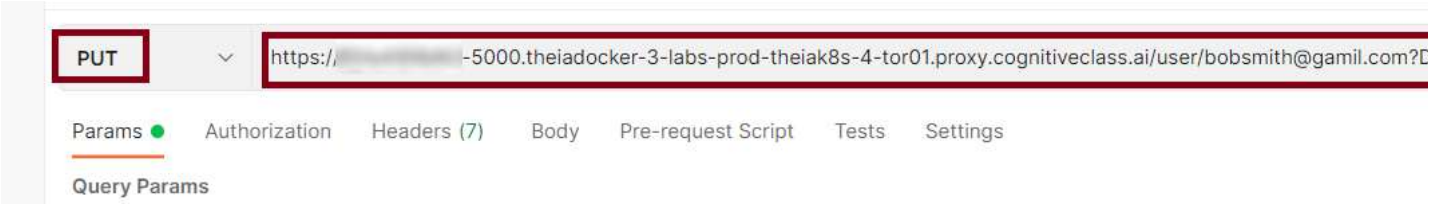
8.4 PUT request

a. Enter the URL by adding the specific email address. If the email address is bobsmith@gamil.com then enter this URL in the input box of the Postman:

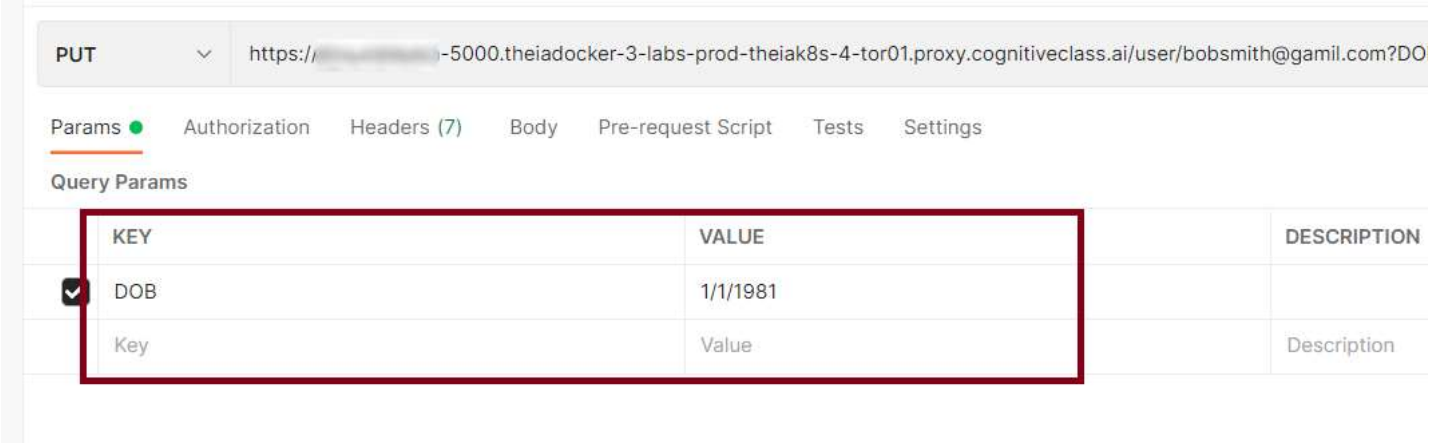
```
1
https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user/bobsmith@gamil.com
```

Copied!

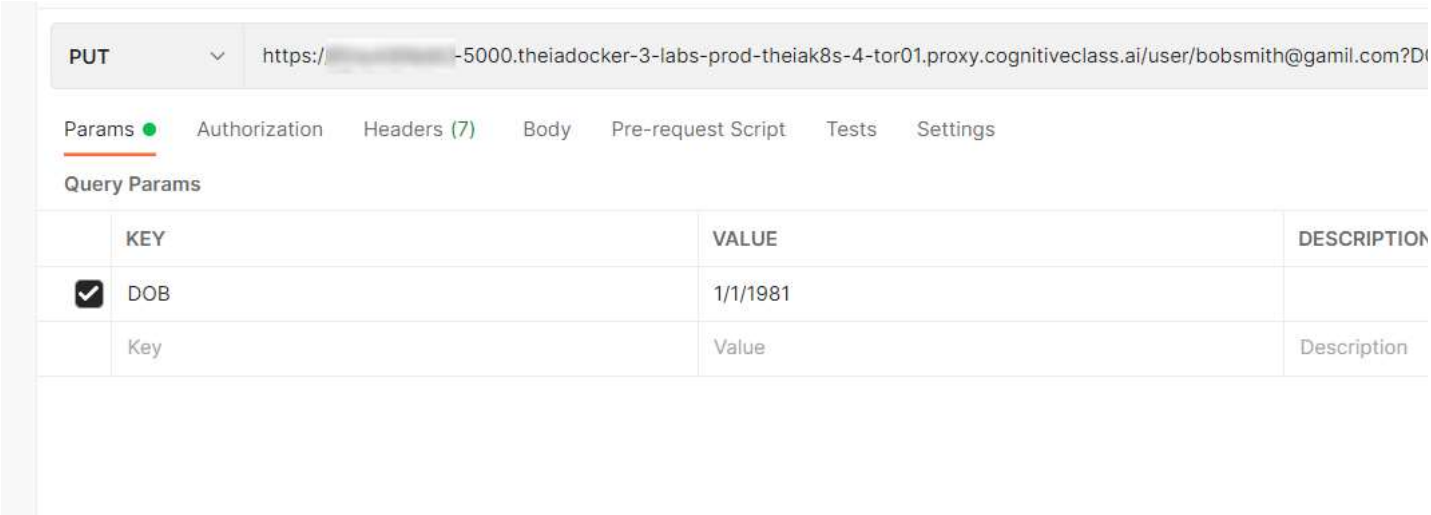
Ensure to select the PUT method and select the "Params".



b. Enter the key and values to be changed. For example, if you want to change the "DOB" key and replace it with the new value 1/1/1981 it will be as below.

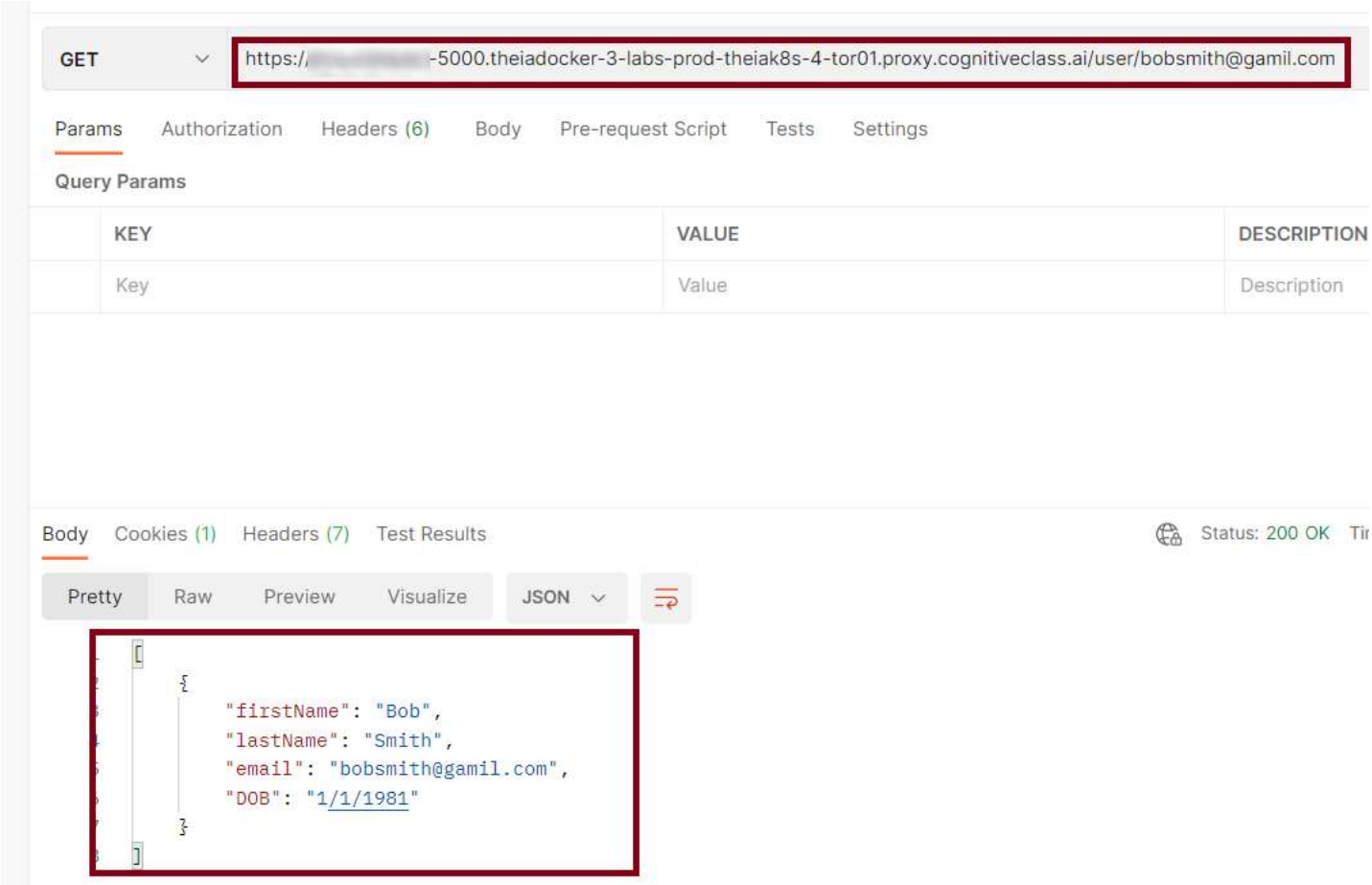


c. Click on the **Send** button after entering the URL to view the output.



Verify that the newly added values are been updated by doing a GET request.

Note: Ensure that you delete any parameters that you added for the PUT request before sending the GET request.



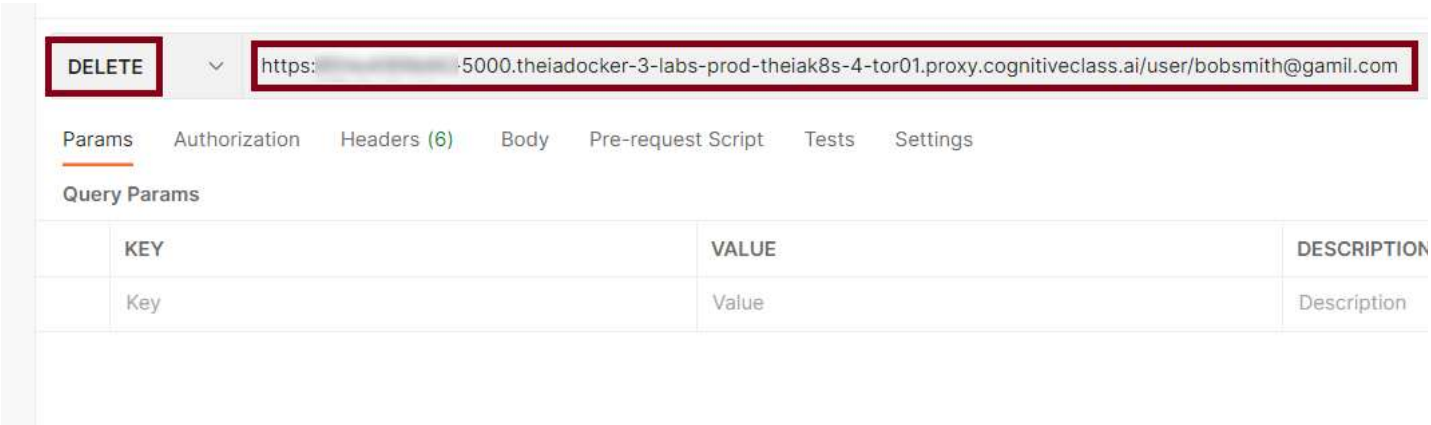
8.5 DELETE Request:

a. Enter the URL by adding the specific email address. If the email address is bobsmith@gamil.com then enter this URL in the input box of the Postman:

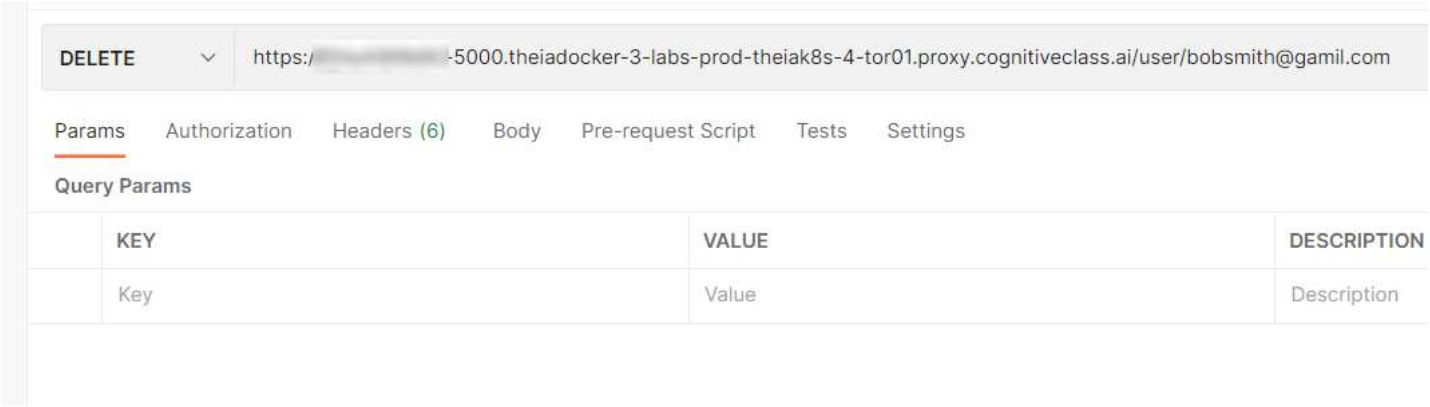
```
1. l
1. https://XXXXXXXXXX-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/user/bobsmith@gamil.com
```

Copied!

Be sure to select the DELETE method.

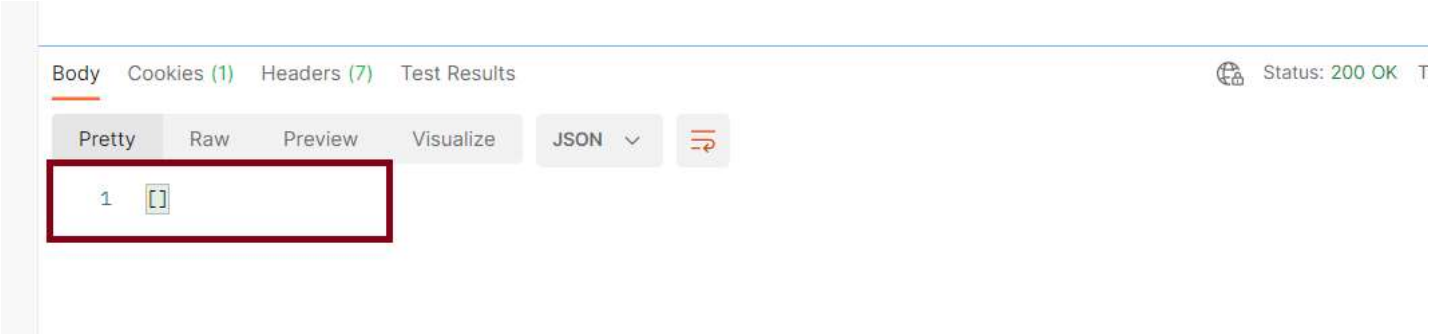


b. Click on the "Send" button after entering the URL to view the output.



c. Verify that the GET user by ID bobsmith@gamil.com returns a null object by sending a GET request.

Note: Ensure that you delete any parameters (if any are there) before sending the GET request.



Practice labs

1. Create an endpoint in the same code for getting all users with a particular Last Name.

▶ Click here for a hint!
▶ Click here for the Solution!

2. Create an endpoint in the same code for sorting users by date of birth.

▶ Click here for a hint!
▶ Click here for the Solution!

Congratulations! You have completed the lab for CRUD operations with Node.js and Express.js using Postman.

Summary:

In this lab, we have performed CRUD Operations like GET, POST, PUT and DELETE on an Express App and tested the above methods using Postman.

Author(s)

Sapthashree K S

K Sundararajan

Changelog

Date	Version	Changed by	Change Description
14-09-2022	1.0	K Sundararajan	Initial version created
30-09-2022	1.1	K Sundararajan	Updated instructions on Author Workbench

(C) IBM Corporation 2022. All rights reserved.